“ राम – राम ”

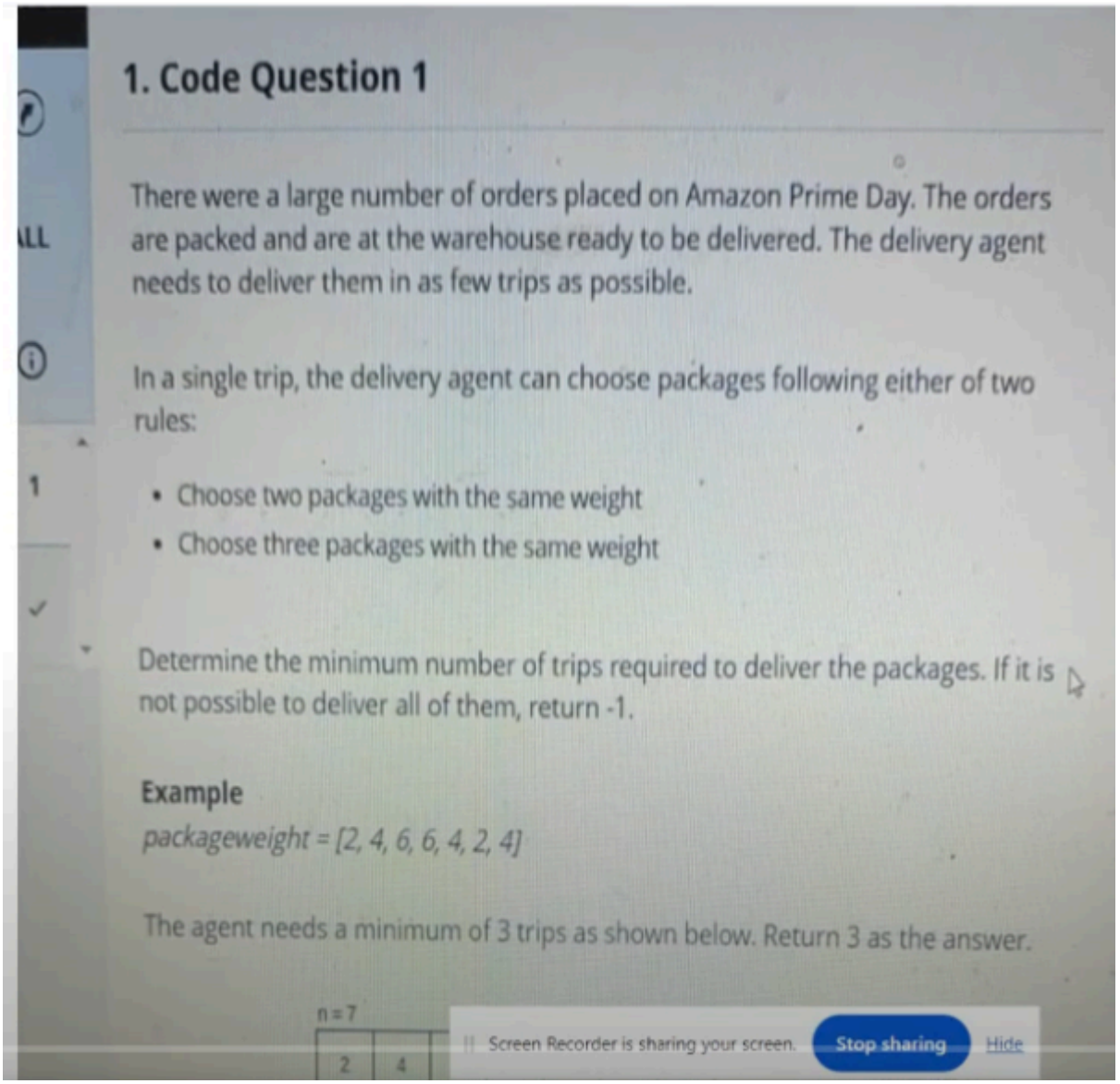
Main Doc :
https://docs.google.com/document/d/1cmjJfoSUh_ng1d2AuLF_Gk07y9RzjOztIOIXwgOLPJU/edit?usp=drivesdk

851 Hours DSA Doc :
https://docs.google.com/document/d/1SosypDZMd652GT3AOHnBDLHdj66ST78-5eysJi7VQq8/edit?usp=drivesdk

“ राम - राम ”

Amazon - OA

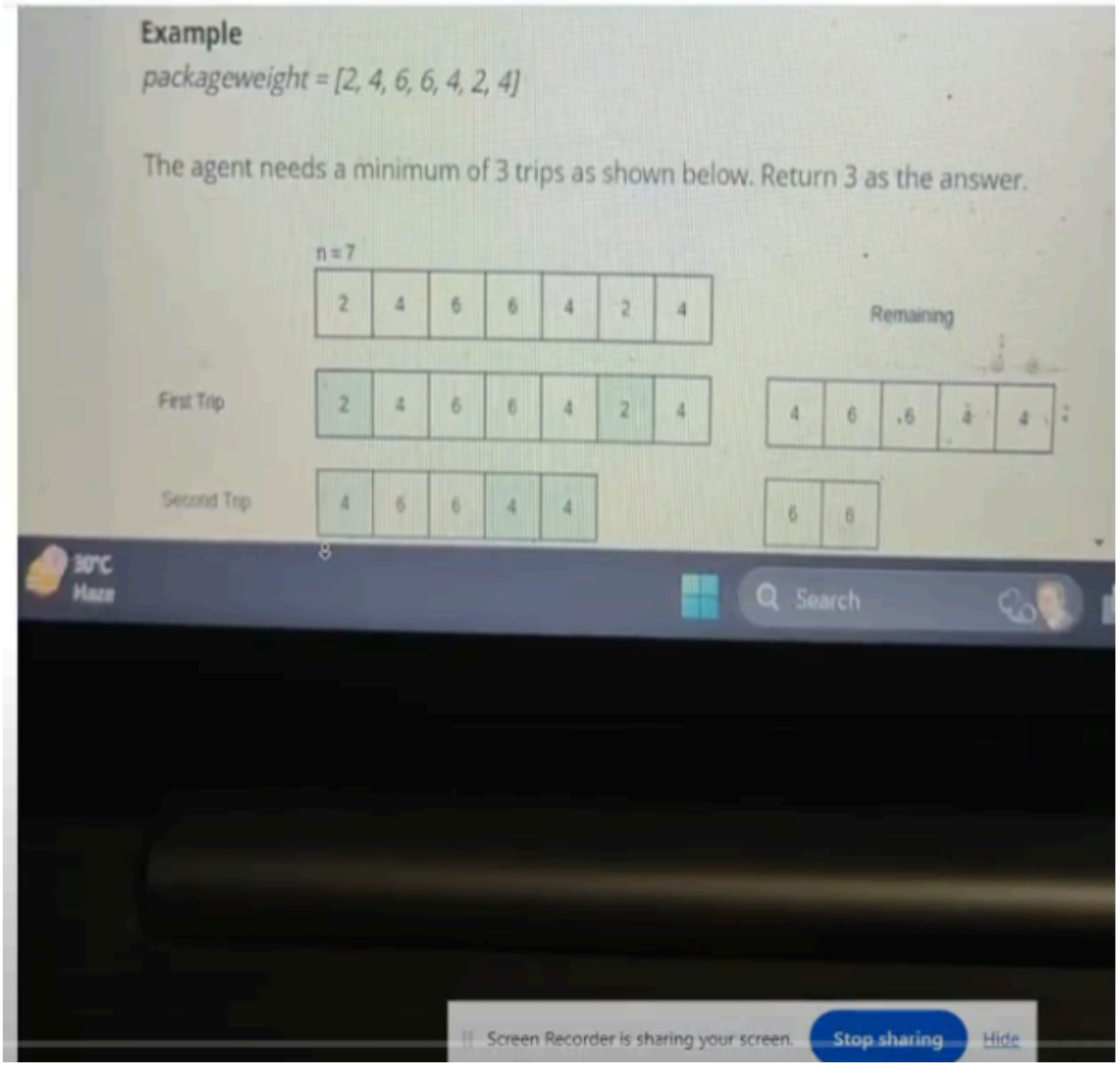
Link
https://www.designa.in/16820/amazon-oa-sde-ctc-45lpa-coding-questions-2024-set-141

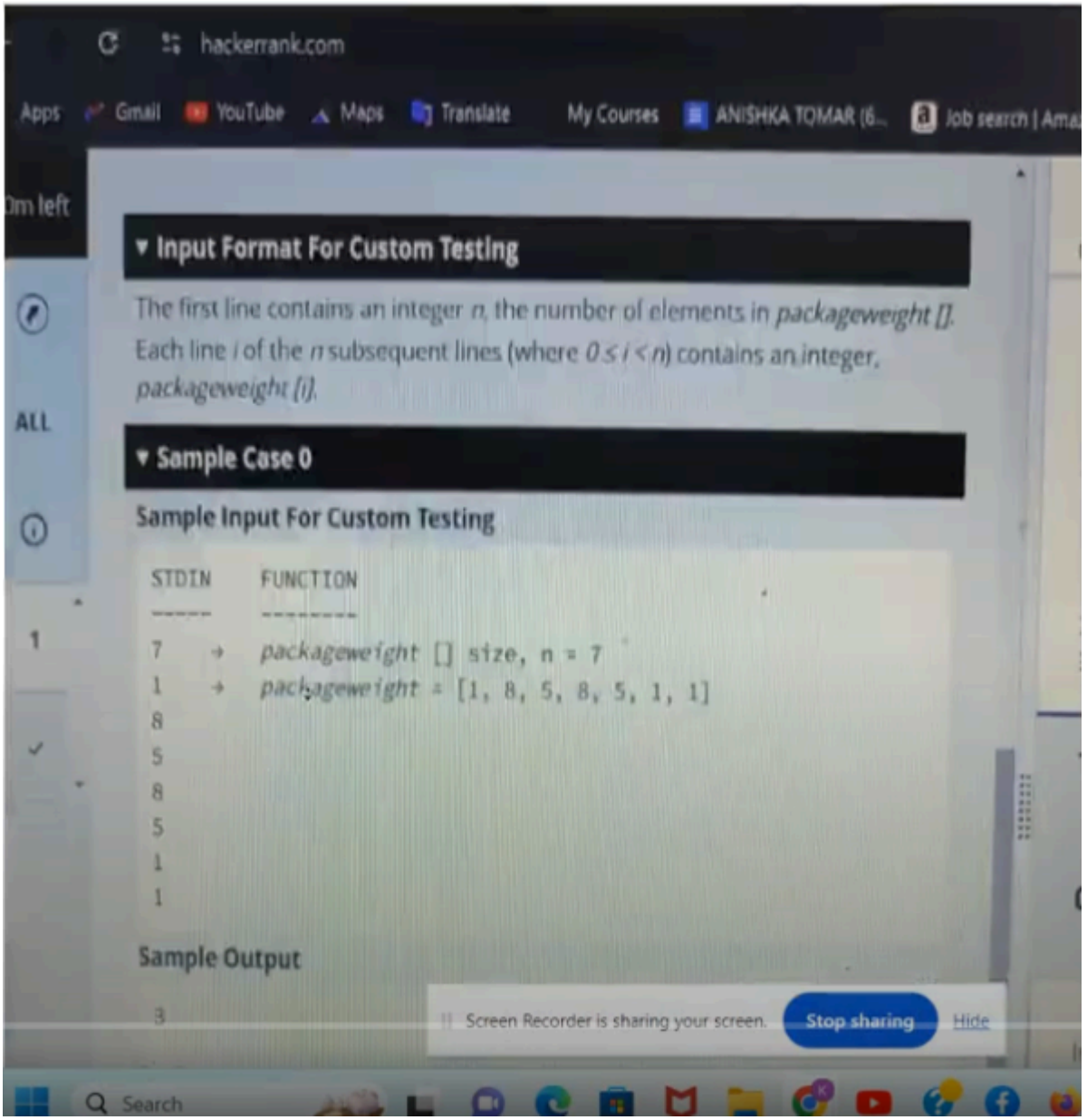


Example

packageweight = [2, 4, 6, 6, 4, 2, 4]

The agent needs a minimum of 3 trips as shown below. Return 3 as the answer.





Understanding
Given an array of size n and we have to deliver packages in minimum trips
In each trip we can choose either of the two options
→ Choose 2 boxes of the same weight
→ Choose 3 boxes of the same weight
Our task is to destroy the whole array and if we are unable to do this then return -1
Ex : [1,8,5,8,5,1,1]
O/p = 3 trips

3 times 1 , 2 times 8 and 2 times 5
Total 3 trips

Example:- [1 1 1 2 2 2 2]

In first move we select 3 - “1”
In second move we select 2 - “2”
In third move we select 2 - “2”

Total 3 moves selects the full array

Let’s say the frequency of a number is coming x times and x is not completely divisible by both 2 and 3 then ?
Say frequency of a number is 32
→ then total trips will be 30/3 + 2/2 = 10 + 1 = 11 and it can be written as 32/3 + 1 = 10 + 1 = 11

Say freq = 31
Trips = 27/3 + 4/2 = 9 + 2 = 11;
Can be written as = 31/3 + 1 = 10 + 1 = 11

Say freq = 22
Trips = 18/3 + 4/2 = 6 + 2 = 8
Can be written as = 22/3 + 1 = 7 + 1 = 8

!!!*** If freq of any number is 1 then return -1
If freq is 2 then 1 trip

Make use of HashMap<>() to store frequencies and check according to conditions
Code : https://onecompiler.com/java/453wqxuck

“ राम - राम ”

Range Update Concept

P0 : You are given an array of size “N”; all elements are 0 in it :)

You are given Q queries :-> [L,R] ; you have to add +1 to all the elements from [L,R]

$1 <= N <= 10^5 / 10^6 / 10^7$

Ex: N= 8

Q = [[1,8], [5,8]]

After all queries are performed we need to print the final array

Sol: Initially → [0,0,0,0,0,0,0,0]

Q[0] = [1,8] → [1,1,1,1,1,1,1,1]

Q[1] = [5,8] → [1,1,1,1,2,2,2,2]

For q[0] = [1,8] ⇒ l = 1, r = 8

So, add a[l] = a[l] + 1 and a[r+1] = a[r+1] - 1

Make use of prefix sum

Brute Force :

T.C = O(N*Q)

Code : https://onecompiler.com/java/453y4aa5j

T.C = O(N+Q)

Google Hackathon Question

We are only solving the easy version of this problem in this session.

1<=a[i]<=100000

1<=k<=100000

a[i]-k>=1 and a[i]+k<=100000

We are given an array of size N and in one operation we can change the any a[i] to a[i]+ x and we can perform only one operation on each a[i] and also x ∈ [-k, k]
such that we get maximum occurrence of same numbers

Ex : [5,8,10] , k = 3
So, we can add any x in [-3, 3] to each a[i] in array such that we get max same numbers

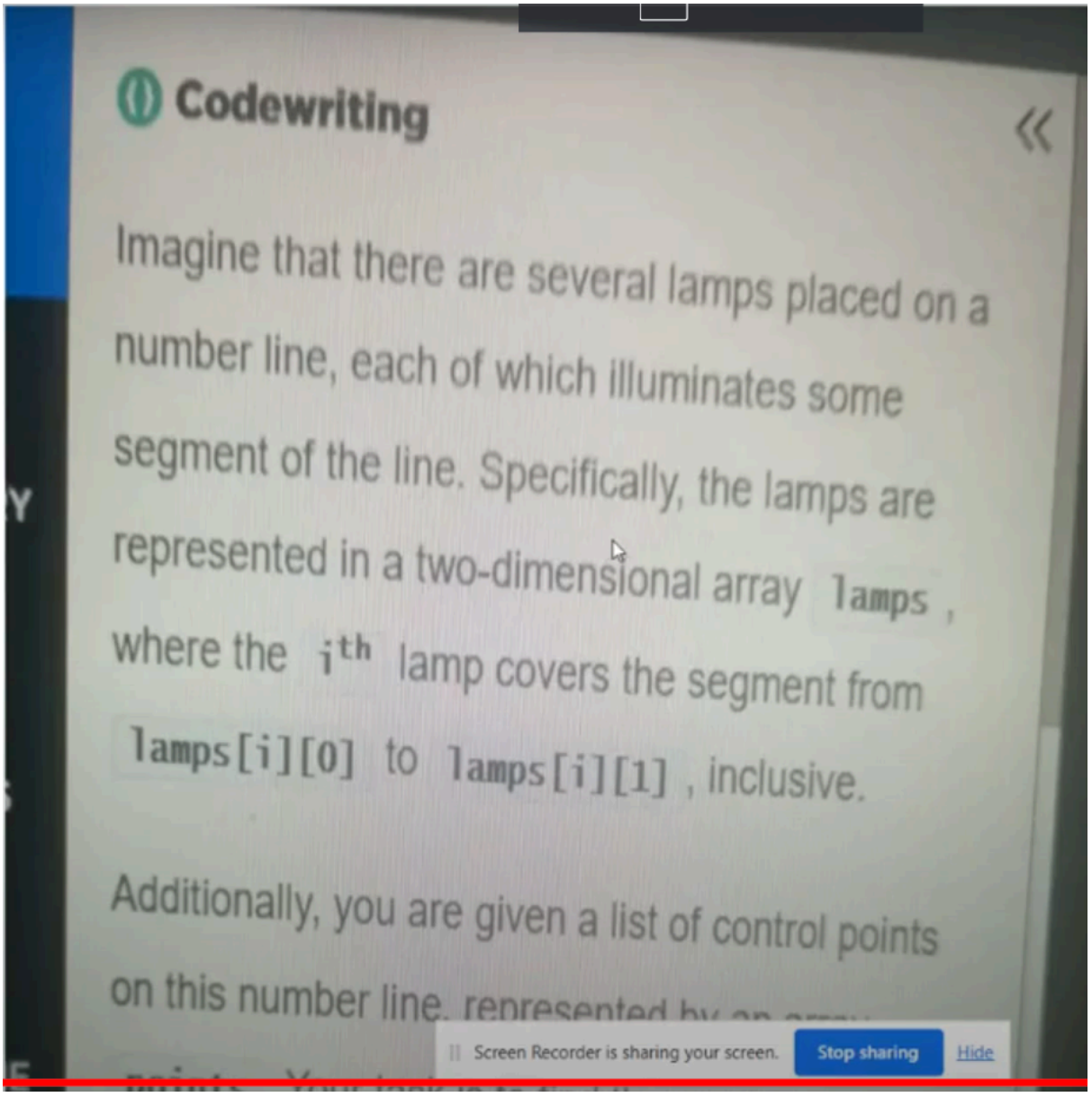
Here we want to find the max occurrence of the number in the array after after adding x to each a[i] such that we get maximum occurrence of a number
When we add -k to a[i] and add k to a[i] we get a range for a[i] out of which we can select a number and check if we getting some max occurrence of that number
Use range update concept in this question such we get a number with max occurrence

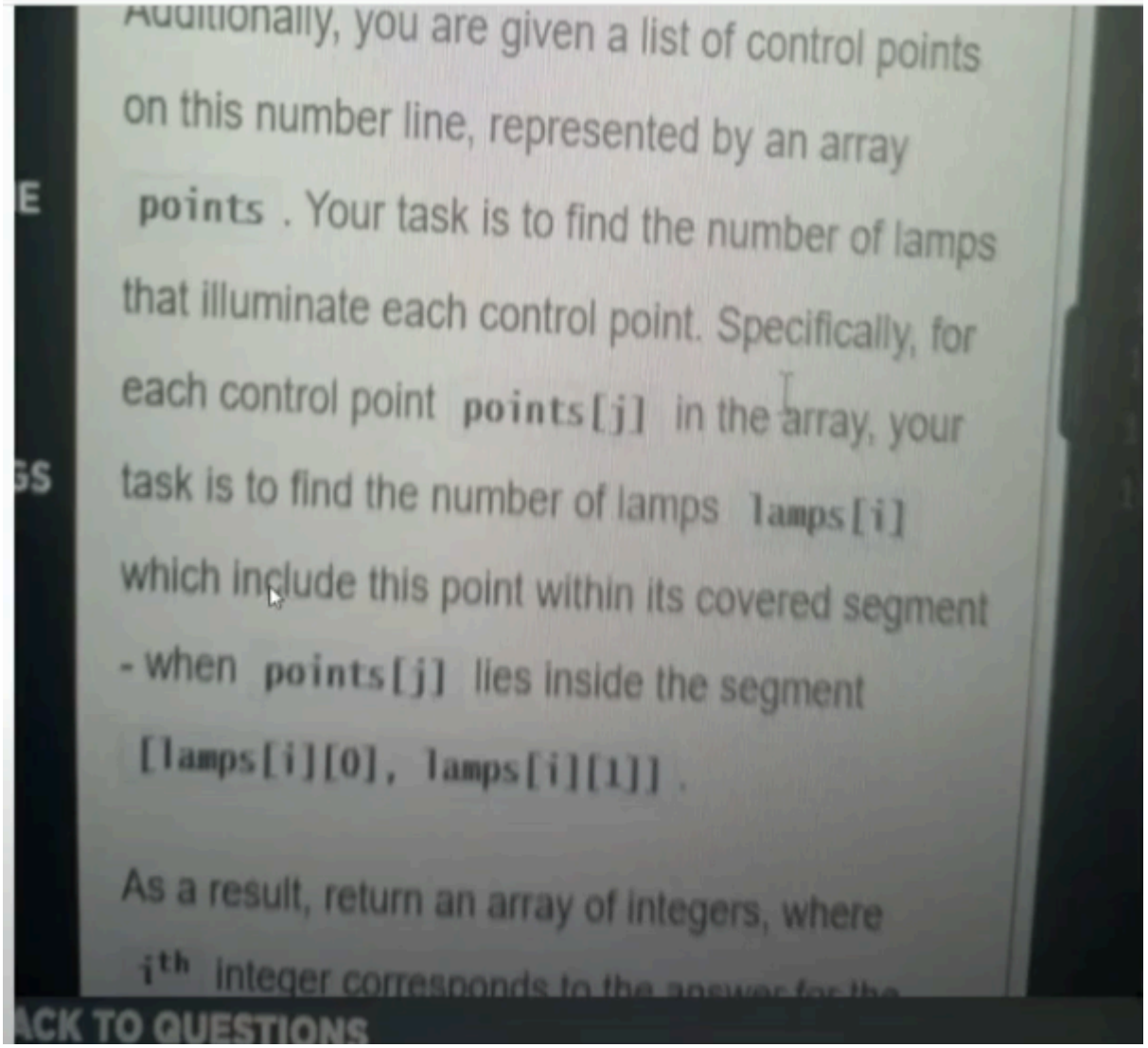
Code : https://onecompiler.com/java/453zgr6q4

" राम - राम "

Link :->

https://www.designa.in/16114/visa-oa-sde-intern-ctc-30-lac-27th-nov



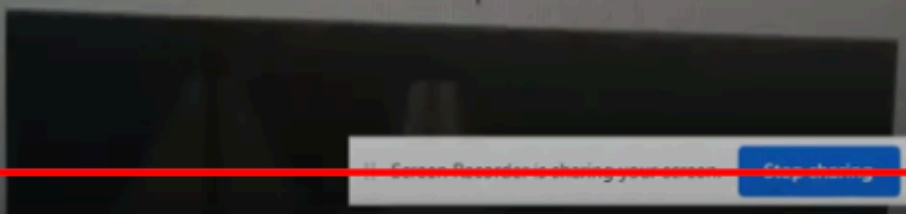

Additionally, you are given a list of control points on this number line, represented by an array points . Your task is to find the number of lamps that illuminate each control point. Specifically, for each control point points[j] in the array, your task is to find the number of lamps lamps[i] which include this point within its covered segment - when points[j] lies inside the segment [lamps[i][0], lamps[i][1]] .
As a result, return an array of integers, where
ith integer corresponds to the answer for the
ACK TO QUESTIONS

Example

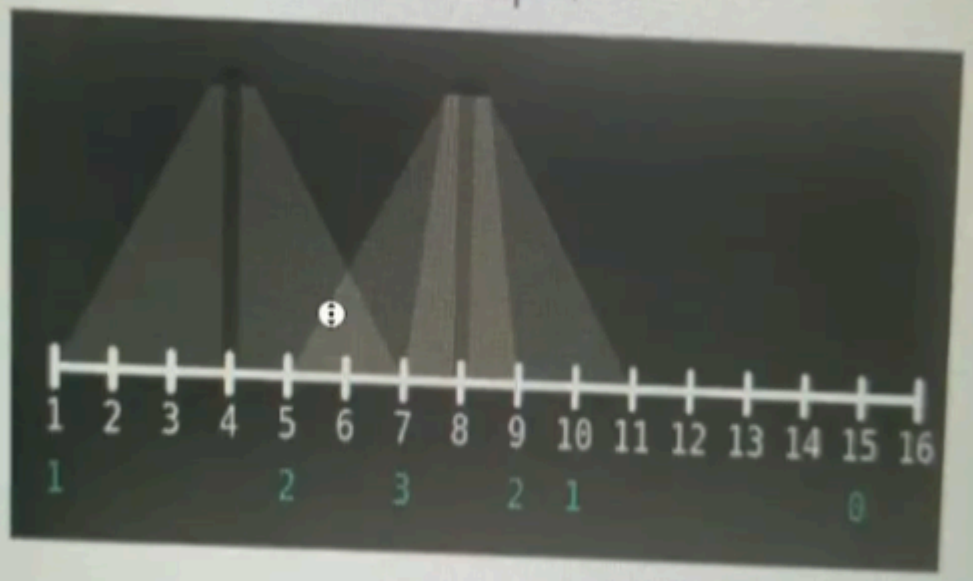
For `lamps = [[1, 7], [5, 11], [7, 9]]` and `points = [7, 1, 5, 10, 9, 15]`, the output should be `solution(lamps, points) = [3, 1, 2, 1, 2, 0]`.

Explanation:

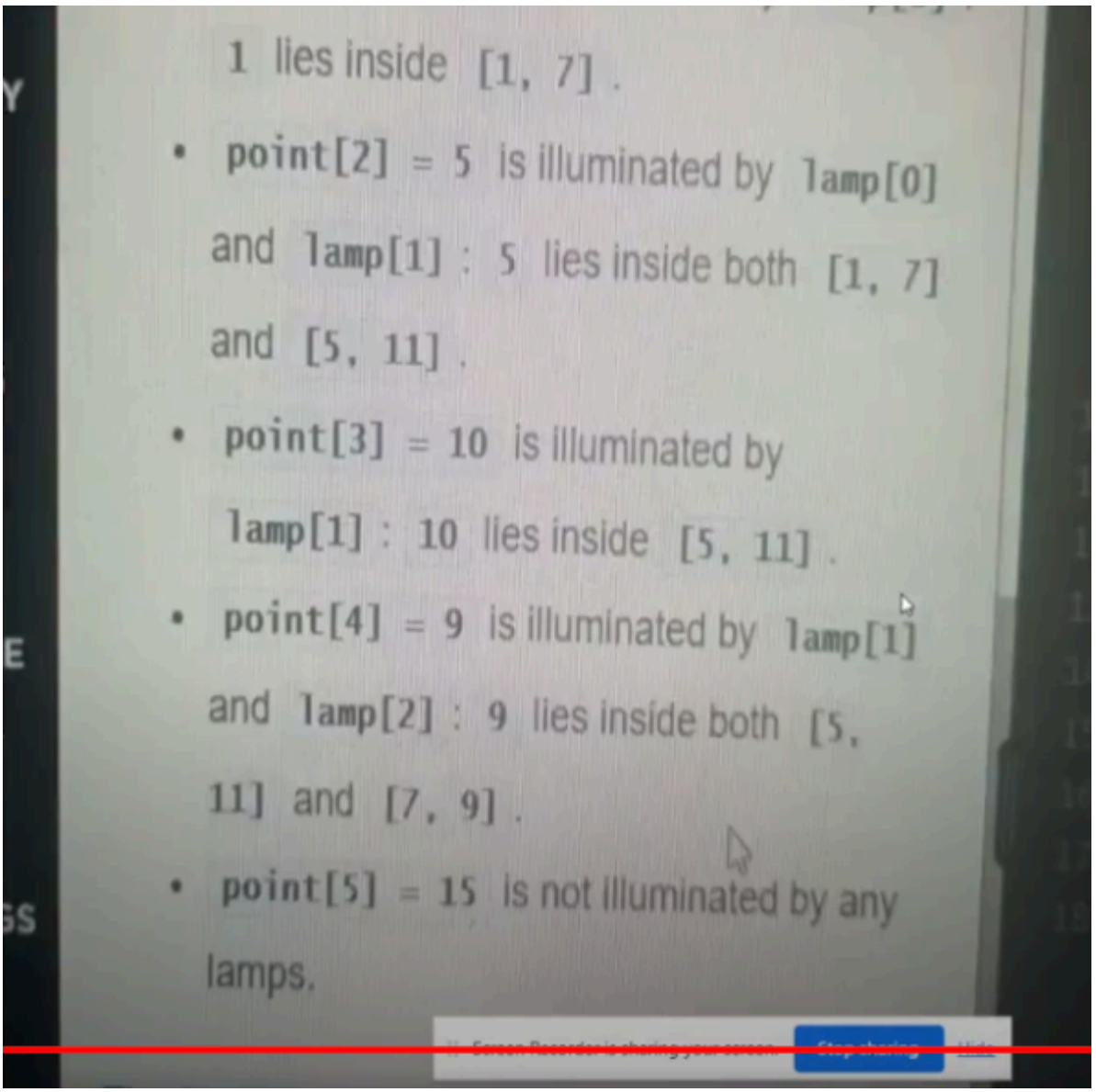
▼ Expand to see the example.



▼ Expand to see the example.



- `point[0]` = 7 is illuminated by all three lamps: 7 lies in all three segments `[1, 7]`, `[5, 11]`, and `[7, 9]`.
- `point[1]` = 1 is illuminated by `lamp[0]`.



$1 <= N <= 10^5$
If $l,r <= 10^5$

Prerequisite:
You are given an array of size N and Query q[i] = (l, r) . You have to add +1 to each query range

Sol → n = 8
Q = {{1, 8}, {5, 8}}
Brute Force :

```
int [] b = new int[n+1]
for(int i=0; i<=n; i++){
        for(int j=q[0]; j<=q[1]; j++){
                b[i] += 1;
        }
}
```

Optimized :

```
int [] b = new int[n+1]
int [] b = new int[n+1]
for(int i=0; i<q.length; i++){
        int l = q[i][0];
        int r = q[i][1]
        b[l] += 1;
        b[r] += -1;
}

for(int i=1; i<=n; i++){
        b[i] += b[i-1]
}
```

Visa Problem :

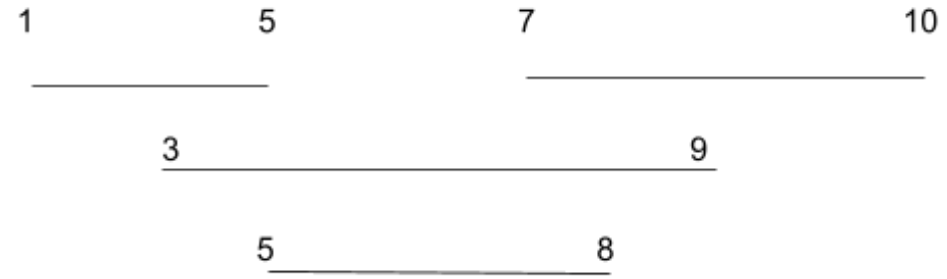
If Constraints were :

$1 <= N <= 10^5$

If $l,r <= 10^5$

Code : https://onecompiler.com/java/454ys7qmp

Ex : [[1, 5], [3, 9], [5, 8], [7, 10]]



Code : https://onecompiler.com/java/454zey6fc

" राम - राम "

You are given an array of size "N" and two integers x and y
Array only contains x and y
Find the count of subarrays which have equal number of x and y
Brute Force : *Use two for loops and count number of x and y if it is equal then count++*

Optimization : Replace x with -1 and y with 1 and this will make our question like count of subarrays whose sum is 0
Code :

Real Question :
You are given an array of size "N" and five integers x, y, w, z ,t
Array only contains x, y, w, z, t
Find the count of subarrays which have equal number of x, y, w, z, t

Let us first solve the easier version of this which is our last question in which two numbers are equal in count subarray
Let us store the count of x in an array and at index j we will check for
$c[j] = c[i-1]$
Let us say we are assuming $x = 1$ *and* $y = 2$
Make prefix count for x and y
Then → if we want the count of x and y to be equal in an subarray ending at index j then
$c_2[j] - c_2[i-1] = c_1[j] - c_1[i-1]$
$\rightarrow c_2[j] - c_1[j] = c_2[i-1] - c_1[i-1]$

So we will store $c_2[i] - c1[i]$ in map and at j we will check if $c_2[j] - c_1[j]$ already exists in the map or not if it exists then we can say that subarray [i ..... j] has equal count of x and y

Code :

Now let us solve for x, y and z
We will store the prefix count of each variable till index index i in three different arrays for each of them
Then we will check at index j if $c_2[j] - c_1[j] == c_3[j] - c_2[j]$

“ राम - राम ”

Link:https://www.desiqna.in/16189/backend-engineer-hackerrank-online-test

Understanding : *We are given an array of size n and where each element contains string representing a person doing some work in the time interval and we want to return the minimum time at which all people are free for k minutes*

Analysis : *We have an array of size n and we want to find the first time at which all the people of the array are free for k minutes*

Observation :

1. *We have to focus only on the time for which a person is doing some work for k*
2. *Convert the given time format in minutes only.*

Code : https://onecompiler.com/java/4546htcmz

“ राम - राम ”

-> Link : https://www.desiqna.in/13820/google-oa-sde2-jan-2023

Task given in problem is to Find Number of Pairs in array A, i and j such that A[A[A[i]]] == A[A[A[j]]]

Ex : [4,2,1,2]

O/p = 6 → (1,2), (1,3), (1,4), (2,3), (2,4), (3,4)

Brute Force :

Check all the pairs whose A[A[A[i]]] == A[A[A[j]]]

Code :

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int n = sc.nextInt();
      int[] a = new int[n+1];

      for(int i=1; i<=n; i++){
        a[i] = sc.nextInt();
      }

      int count = 0;
      for(int i=1; i<a.length; i++){
        for(int j=i+1; j<a.length; j++){
          if(a[a[a[i]]] == a[a[a[j]]]) count++;
        }
      }

      System.out.println(count);
    }
}
```

Optimized : Make use of map in which we will store a[a[a[i]]] and ts frequency

Code : https://onecompiler.com/java/454q4ka69

“ राम - राम ”

Link Media.Net OA | SDE1 | CTC 28 L - Desi QnA

“ राम - राम ”

Link :
https://www.desiqna.in/13736/arcesium-oa-sde1-jan-2023

You are given two string arrays a and b and we want to print yes if we can make a[i] and b[i] twins by performing some operations
Operations :

1. Swap a character at any even index with any other even index
2. Swap a character at any odd index with any other odd index

Before solving this question we can solve an easier version
Let us say we have two strings A and B , we can swap any character with any other in the string and can we make them equal by some swaps
A = “bghy”,
B = “ghby”

Observation :
If we can swap any character with any other character then index doesn't matter only the freq matters of that character in both the strings

Create map1 for string A and map2 for string B
String a , b;
if(a.length() != b.length()) print(No);
HashMap<Character, Integer> map1;
HashMap<Character, Integer> map2;

```
for(int i=0; i<a.length(); i++){
        map1.put(a[i], map.getOrDefault(a[i], 0)+1);
}

for(int i=0; i<b.length(); i++){
        map1.put(b[i], map.getOrDefault(b[i], 0)+1);
}

if(equal(map1, map2)) print("Yes");
else print("No")

public boolean equal(map1, map2){
        for(Character key : map1.keySet()){
        Int freq1 = map1.get(key);
        if(map2.containsKey(key)){
                Int freq2 = map2.get(key);
                if( freq1 !=  freq2) return false;
        }
        else return false;
        }
        return true;
}
```

T.C = O(N)
S.C = O(1)

Real Question :
In this we have to do swapping of even index only
With even index and odd index with odd index

We don;t want to do anything with order we just want to see the quantity of the each character so thats why we are using map in this
Code: https://onecompiler.com/java/454ahws4e

“ राम - राम ”

Given four integer arrays a, b, c, d having length n and we have to find all the tuples(i, j, k, l) such that

0 <= i, j, k, l < n

a[i] + b[j] + c[k] + d[l] = 0

1 <= n <= 200

-228 <= a[i], b[i], c[i], d[i] <= 228

Brute Force :

We will use 4 nested loops and compare all possible combinations

Code : https://onecompiler.com/java/454ayrh8h

T.C =O(N^4), S.C = O(1)

Optimized: T.C = O(N^3)

In this what we will do is → store the -(d[l]) in the map and then using 3 nested loops we will check for

(a[i] + b[j] + c[k]) in the map if it is present increase the count

a[i] + b[j] + c[k] + d[l] = 0

a[i] + b[j] + c[k] = -d[l]

Code : https://onecompiler.com/java/454b7hxkq

T.C = O(N^3), S.C = O(N)

Further Optimization : T.C = O(N^2), S.C = O(N^2)

Before we were storing d[l] in map now we will store c[k] + d[l] in map

a[i] + b[j] = -(c[k] + d[l])

T.C = O(N^2), S.C = O(N^2)

Code :

“ राम - राम ”

Given an array of size n and we have to find the number of triplets such that A[i] > A[j] < A[k]
i < j < k, 1 <= n <= 1000
Ex: [8, 1, 2, 3, 4, 5]
O/p = 10 → (8,1,2), (8,1,3), (8,1,4), (8,1,5), (8,2,3), (8,2,4), (8,2,5), (8,3,4), (8,3,5), (8,4,5)

Brute Force :
Code : https://onecompiler.com/java/454d79dxn
T.C = $O(N^3)$, S.C = O(1)

Optimized :
First see how we can find the pairs (i, j) such that a[i] > a[j]
By using 2 for loops we will store the count of all the numbers greater than a[j] in pre[j] from i= 0 to j-1 for each j

```
for(int j=1; j<n; j++){
        int count = 0;
        for(int i=0; i<j; i++){
                if(a[i] > a[j]) count++;
        }
        pre[j] = count;
 }
```

Similarly we will count all the numbers greater a[j] from j+1 to n in suff[j] for all j

```
for(int j=n-2; j>=0; j--){
        Int c = 0;
        for(int k=n-1; k>j; k--){
```

```
            if(a[j] < a[k]) c++;
        }
        suff[j] = c;
}
```

Now we have pre[j] storing numbers greater than j till j-1
And suff[j] storing numbers greater than j from last
So we can get get ans by by multiplying pre[j]*suff[j]
Code: https://onecompiler.com/java/454d9sxdb
T.C = $O(N^2 + N^2 + N) = O(N^2)$
S.C = O(N + N) = O(N)

Q. Given an array of size n and we have to find the number of quadruplets (i, j , k, l) such that A[i] > A[j] < A[k] > A[l]
1 <= n <= 1000

" राम - राम "

Given 5 arrays :- A,B,C,D,E all of size "N"
Element in the array can range from [-10^9.....10^9]
Find the number of tuplets (i,j,k,l,m) such that -
a[i] + b[j] + c[k] + d[l] + e[m] = 0

A = [2 5]
B = [3 8]
C = [-5 8]
D = [5 10]
E = [-10 100]
Output - 2 → (2,5,-5,10,-10) , (2, 3, -5, 10, -10)

Brute Force :

```
Int count = 0;
for(int i=0; i<n; i++){
      for(int j=0; j<n; j++){
          for(int k=0; k<n; k++){
              for(int l=0; l<n; l++){
                  for(int m=0; m<n; m++){
                      if(a[i] + b[j] + c[k] + d[l] + e[m] = 0)
count++;
                  }
              }
          }
      }
}
```

T.C = $O(N^5)$, S.C = O(1)

Optimized :

If we want a[i] + b[j] + c[k] + d[l] + e[m] = 0
Then we can store elements of array e in a map with negative sign a[i] + b[j] + c[k] + d[l] = - e[m]

Code : https://onecompiler.com/java/454fyby6r
T.C = O(N^4), S.C = O(N)

Optimization : We can store -(d[l] + e[m]) in hashmap
B'coz → a[i] + b[j] + c[k] = - (d[l] + e[m])
Code : https://onecompiler.com/java/454fypc85
T.C = O(N^3) , S.C = O(N^2)

“ राम - राम ”

You are given an array of strings. For each string in the array, calculate the sum of matching characters between that string and every subsequent string in the array. Return an array containing the sum of matching characters for each string, where each element at index i in the array corresponds to the sum of matching characters of the ith string with every subsequent string.

Two characters are considered matching if they are the same and occur at the same index in both strings. If the length of the two strings being compared is not equal, only check up to the length of the shorter string.

Input:
N = 3
X = {"abc", "ade", "bbc"}
Output: [3, 0, 0]
Explanation:
For String "abc"(i = 1), the subsequent strings are "ade" and "bbc". The matching characters of "abc" with "ade" is 1 which is 'a' at index 1 and the matching characters of "abc" with "bbc" is 2 which are 'b' and 'c' at index 2 and 3. The answer for "abc" is 1 + 2 = 3
For String "ade"(i = 2), the subsequent string is "bbc". The matching characters of "ade" with "bbc' is 0. The answer for "ade" is 0.
For String "bbc"(i = 3), there are no subsequent strings. The answer for "bbc" is 0.

We can note that the answer for the last string in the input array will always be 0 as there are no subsequent strings.

Understanding : We are given a string array and we want to find the sum of all matching characters of current string with the subsequent strings
Matching character means it should be same and should have the same index

If we are at index i then we have to store sum of all matching characters with strings at i+1, i+2, ... , n-1 at index i
Ex : N = 5
arr = [“bbz”, “zaz”, “aaa”, “zaa”, “zzz”]

ans = [2, 5, 2, 1, 0]

Example **2:**

Input:
N = 3
X = {"abc", "ade", "abc"}
Output: [4, 1, 0]
Explanation:
For String "abc"(i = 1), the subsequent strings are "ade" and "abc". The matching characters of "abc" with "ade" is 1 which is 'a' at index 1 and the matching characters of "abc" with "abc" is 3. The answer for "abc" is 1 + 3 = 4
For String "ade"(i = 2), the subsequent string is "abc". The matching characters of "ade" with "abc' is 1. The answer for "ade" is 1.
For String "abc"(i = 3), there are no subsequent strings. The answer for "abc" is 0.

1<=N<=10000
Sum of all string length<=100000

Brute Force :
For each string in the array we will find the sum of all matching characters of its subsequent strings
Code : https://onecompiler.com/java/454geuymh

“ राम - राम ”



Find the increasing subsequence of size 3 if it exists return true otherwise false

-> [18 5 4 3 2 1 8 10]

O/p = True → (1,8,10), (2,8,10), (3,8,10), (4,8,10), (5,8,10)

Brute Force : In this we will make use of 3 for loops and will check for a[i]<a[j]<a[k]

```
for(int i=0; i<n; i++){
        for(int j=i+1; j<n; j++){
                for(int k=j+1; k<n; k++){
                        if( a[i]<a[j] && a[j]<a[k] ) return true;
                }
        }
}
return false;
```

T.C = $O(N^3)$, S.C = O(1)

Optimization : If we want to optimize it we can make use of suffix array with the help of which we can easily store the max element at i+1 in (i+1 ..... n-1) for element at i index

```
int[] suff = new int[n]
Suff[n-1] = arr[n-1]
for(int k=n-2; k>=0' k--){
        suff[k] = Math.max(arr[k], suff[k+1])
}
for(int i=0 i<n; i++){
        for(int j=i+1; j<n; j++){
                if(a[i]<a[j] && a[j] < suff[j+1]){
                        return true;
                }
        }
}
Return false;
```

T.C = $O(N^2)$, S.C = O(N)

Optimization :

In order to get the increasing subsequence we need to check if we are at i index then if we know the smallest till i-1 and max element among (i+1 ..... n-1) then we can easily say that there is an increasing subsequence possible with index i
Smallest can be stored in the prefix array and largest can be stored in the suffix array
Code : https://onecompiler.com/java/454h4xhyv

HW :- Same as above question but the triplet should be of the form:- (b[i]+1==b[j] && b[j]+1==b[k]) => b[i]<b[j]<b[k]

" राम - राम "

Find the number of ways to divide the array into two equal parts

Question to be Asked :

→ *What can be the max size of the array and does it contain negative elements*

1 <= n <= 100000, there can be negative elements in the array

Ex : [1, 2, 0, 0, 1, 2]

O/p = 3 →

Observation 1 : In order to divide the array, total sum should be even

Since we want to divide the array into two equal parts

So we will put a stick on each index and and calculate the LHS and RHS, if LHS = RHS then we will say that, we can divide array into two equal parts at that index

LHS = sum[0 ..... i-1] + a[i]

RHS = totalSum - LHS

OR we can check

If → LHS = totalSum/2

```
int totalSum = 0;
for(int i=0; i<n; i++){
        totalSum  += a[i]
}

int lhs = 0;
for(int i=0; i<n; i++){
        totalSum  += a[i]
```

```
}
Int ways = 0;
for(int i=0; i<n; i++){
        Lhs += a[i]
        if(Lhs == totalSum/2) ways++;}
```

Code : https://onecompiler.com/java/454jrqztu

T.C = O(N + N) = O(N)

S.C = O(1)

Follow up :

Find the number of ways to divide the array into three equal parts

TotalSum % 3 = 0 → TotalSum = 3y

Each part = y

If we want to find the number of ways to divide the array into three equal parts then

LHS = sum[0 ..... i] = 2y

But the question is how you will count the partitions for this

We can make use of prefix sum and map in which we will store the prefix sum and at index i we will check for map.get(pre[i]/2) If it present then we have found the subarray having sum 2y

Code : https://onecompiler.com/java/454kpf2g4

Real Question

Find the number of ways to divide the array into four equal parts

First of all totalSum % 4 == 0

Each part = y

If we want to divide the array into four equal parts then if at index i e get the prefix sum or LHS to be 2y then it is sure that RHS = 2y

if we are able to find the number of ways to divide the lhs into two equal parts and rhs in two equal parts then our answer of partition at this index will be multiplication of (lhs*rhs)

Code : https://onecompiler.com/java/454ppqzqe

" राम - राम "

Understanding :-> We are given an array of size "N". We have to pick "x" elements at exactly "y" distance; sum of all them should be minimum.

→ [1 5 8 1 4 5 8], X = 2; Y =2;
Possible Pairs : (1, 8), (5, 1), (8, 4), (1, 5), (4, 8) ⇒ Min = (5, 1) = 6

→ [1, 2, 5, 1 , 8, 8, 4], X = 2, Y = 3
Possible Pairs : (1, 1), (2, 8), (5, 8), (1, 4) ⇒ Min = (1, 1) = 2

→ [1, 2, 5, 1 , 8, 8, 4, 2, 3], X = 3, Y = 2
Possible Pairs : (1, 5, 8), (2, 1, 8), (5, 8, 4), (1, 8, 2), (8, 4, 3)
⇒ Min = (2, 1, 8) = 1 1

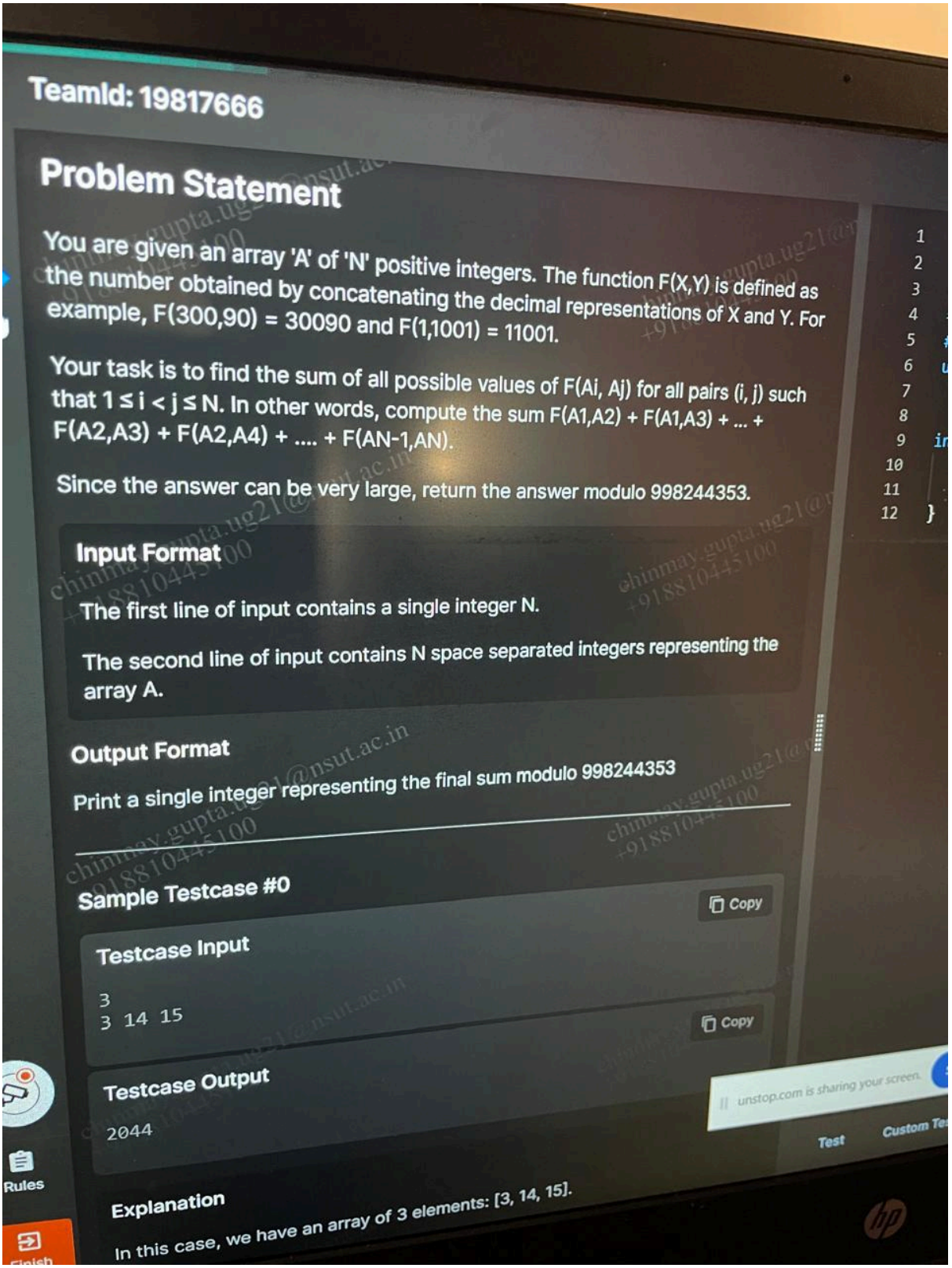
Brute Force :
Code : https://onecompiler.com/java/4556dd24q

Optimized :
In this question we have to take only those x who are at a distance of y from each other
*Somehow if we are able to store the sum of all the x at a distance of y from index 1 till index i then we can easily tell the sum of last x from current index by subtracting the sum of prefix[i - x*y]*

*pre[i] = a[i] + a[i-y] + a[i-2*y] + a[i-3*y] + ... + a[1]*

Code : https://onecompiler.com/java/4556frgzy

“ राम - राम ”



Understanding - For all pairs (i,j) such that i < j calculate F(i,j) and find sum of all the F(i,j) | 1 <=N <= 10^5 | 1<=B[i]<=10^5

We have to find the sum of all pairs (i , j) according to the function
F (i , j) = a[i] . append(a[j]) → F (13, 302) = 13 + 302 = 13302
Brute Force :
https://onecompiler.com/java/454qcm5vt

Optimized :
Analysis : when we want to concatenate two numbers then if we somehow able to find a way to concatenate numbers
f(13, 302) = 13*1000 + 302 = 13302
f(b[i], b[j]) = $b[j]*10^{len(b[j])}$ +b [j]
So if we want to find the sum of all pairs of i with some index j then
F[1, b[j]] + f[2, b[j]] + ... +f[i-1, b[j]] = b[j] * (j-1) + $pre[i-1]*10^{len(b[j])}$

“ राम - राम ”

Link to question:->
https://www.desiqna.in/16227/uber-oa-sde-1-ctc-65-l-jan-2024.



We are given an array of size n and we have to find calculate the sum of every possible pair a[i] * a[j] where a[i] * a[j] represents string concatenation of a[i] , a[j] resp.

Ex : [10,2]

Pairs → a[0] . a[0] = 1010

a[0] . a[1] = 102

a[1] . a[0] = 210

a[1] . a[1] = 22

Total sum = 1344

Brute Force :

Code : https://onecompiler.com/java/454rmc8az

Optimization :

We want the sum of all possible pairs

Let's first calculate the sum of all possible pairs from [1 ……j]
Now if we want to calculate sum efficiently then if we see it carefully
We want to append the a[j] to a[i] then we want a[i] to get multiply by $10^{len(a[j])}$ so that we can get appended result
Ex : (13, 421) = 13421
⇒ $13*10^3 + 421 = 13000 + 421 = 13421$

F[a[1] , a[j]] + F[a[2], a[j]] + ... + F[a[j-1], a[j]]
= a[1] . a[j] + a[2] . a[j] + ... + a[j-1] . a[j]
= (a[1] * $10^{len(a[j])}$ + a[j]) + (a[2] * $10^{len(a[j])}$ + a[j]) + ... +
(a[j-1] * $10^{len(a[j])}$ + a[j])
= $10^{len(a[j])}$(a[1] + a[2] + ... a[j-1]) + a[j] * (j-1)
= $10^{len(a[j])}$ * prefixSum[1 ..... j-1] + a[j] * (j-1)

Similarly we have to do this with the suffix so that we can get all the reverse pairs
To get the reverse pairs sum we will calculate the suffix sum and use the same formula as we have used for prefix with small changes
If we are at index j then
→ suffix[i] = suffix[i+1] + a[i]
→ pairSum = $10^{len(a[j])}$ * suffixSum[j+1 ..... n] + a[j] * (n-j)

Code : https://onecompiler.com/java/454rruq9e

“ राम -राम ”

Link : Google OA | SWE Intern | July 2023 | Solution by Kumar K - Desi QnA

Prerequisite Problem! -> https://leetcode.com/problems/max-number-of-k-sum-pairs/description/

Understanding of the problem

We are given an array of size n and we have to find the maximum number of pairs whose sum is k and we can not pick the elements of this pair to pair up with any other pair i.e

Once we select pair (x, y) then we can not choose x, y for any other pair

Ans → We have to find the pairs whose sum is k

So we can store all the elements of the array in map with their frequency

Now we will iterate through this map and for each key we search for k-a[i] if it exists then we take

min(freq(a[i]), freq(k-a[i])) to see the count of pairs

If a[i] = k - a[i] then we can say number of pairs will freq/2

Code :

" राम - राम "

P0 link
https://leetcode.com/contest/weekly-contest-365/problems/maximum-value-of-an-ordered-triplet-i/

Understanding :
We are given an array of size n and we have to find the max value of (a[i] - a[j]) * a[k] for ordered triplet (i, j, k)

Brute Force :
*T.C = O(N*N*N)*
S.C = O(1)
Code :
https://leetcode.com/contest/weekly-contest-365/problems/maximum-value-of-an-ordered-triplet-i/submissions/2158700235/

Optimized :
*T.C = O(N*N), S.C = O(N)*
In this we will make use of pre[k] array so that we can store the max a[i] - a[j] till index k-1 getting stored at pre[k] by chec
Code :
https://leetcode.com/problems/maximum-value-of-an-ordered-triplet-ii/submissions/2159468238/

Further Optimization:
*Since we want the max (a[i] - a[j]) * a[k]*
So if we are able to find the max a[i] - a[j] till k-1 and we also have to maintain a max to store max a[i] till k-1 so we can get a max pair (i, j) for k and we will do this for each k and take max out of all

Code:
https://leetcode.com/problems/maximum-value-of-an-ordered-triplet-ii/submissions/2159531884/

“ राम - राम ”

Google Question :

→ Count the number of mountain subarrays

Ex : [1, 2, 4 , 2 , 1]

O/p → [1,2,4,2], [1,2,4,2,1], [2,4,2], [2,4,2,1] ⟹ Total = 4 subarrays

Brute Force :

Q. Calculate the longest increasing subarray

→ For this we will make use of pre[i]

Pre[i] = store the length of longest strictly increasing subarray ends at index i

For this [1, 2, 4 , 2 , 1]

→ pre[] = [1, 2, 3, 1, 1]

Ans = max(pre[i], pre[i+1], ... , pre[n-1])

Q. Calculate the longest decreasing subarray

→ For this we will make use of suff[i]

suff[i] = store the length of longest strictly decreasing subarray starts at index i

→ [1, 2, 4 , 2 , 1]

suff[] = [1, 1, 3, 2, 1]

Ans = max(suff[i], suff[i+1], ... , suff[n-1])

Q. Find the largest mountain array

Ans = max(pre[i-1] + suff[i+1] - 1)

Q. Calculate the number of mountain subarrays whose topmost index is ‘ i ’

To calculate number of mountain subarrays at index i pre[i-1] * suff[i+1]

pre[i-1] not only represents the longest increasing subarray but it also represents the number of valid indexes from which our subarray starts

Now we can easily solve the google question by calculating the mountain array at each index

[1, 2, 4 , 2 , 1]

pre[] = [1, 2, 3, 1, 1]

suff[] = [1, 1, 3, 2, 1]

" राम - राम "

Q:- Given an array of N positive integers; find the number of valleys ; of size >=3

$1 <= N <= 10^5$

Ex : B = [5 3 4 8]

→ (5, 3, 4) , (5, 3, 4, 8)

Ex : B = [5, 3, 1, 8, 10]

→ (5, 3, 1, 8), (5, 3, 1, 10), (3, 1, 8), (3, 1, 8, 10)

Brute Force :

Sol : In this we are checking valley at each index between array [i ..... j] i.e we are checking each valley condition in each and every subarray

Code : https://onecompiler.com/java/455abg4bz

Optimized:

" राम - राम "

We are given an array of size n; for each index i output the subarray sum which is largest and ends at index i

→ [5, -8, 1, 1, 5]

O/p ⟹ ans = [5, -3, 1, 2, 7]

For i = 0 ⟹ max sum possible → ans[0] = a[0]

For i = 1 ⟹ max sum possible → ans[1] = max(a[1], a[1] + ans[0])

For i = 2 ⟹ max sum possible → ans[2] = max(a[2], a[2] + ans[1])

.

.

.

For i=n-1 ⟹ max sum possible → ans[n-1] = max(a[n-1], a[n-1] + ans[n-2])

⟹ ans[i] = max(a[i], a[i] + ans[i-1])

Find max sum of two non overlapping subarrays

→ Brute Force :

Make use of 4 for loops initial 2 for checking each subarray and next 2 loops for max second subarray

```
int max = 0;
for(int i=0; i<n; i++){
    int sum = 0;
    for(int j=i; j<n-1; j++){
        sum += a[j];
        int maxSecond = Integer.min_value
        for(int k=j+1; k<n; k++){
```

```
                int sum2 = 0;
                for(int l=k; l<n; l++){
                sum2 += a[l];
                maxSecond = Math.max(maxSecond, sum2);
            }
        }
        max = Math.max(max, maxSecond)
}

Ans = max
```

Optimized

Let;s first fix the first array

The first array can be an array ending at index 0, index 1, ……., index n-1

So let's calculate the pre[i] which will tell us the max subarray sum ending at index i

```
n
a[n+1]
pre[n+1];
for(int i=1; i<=n; i++){
        pre[i] = max(a[i], a[i] + pre[i-1]);
}
```

Now let's calculate the suffix array

```
s[n+1]
```

s[j] ⟹ it will tell us that max possible subarray sum starting at index j

```
 for(int i=n-1; i>=2; i++){
        s[i] = max(a[i], a[i] + s[i+1]);
}
```

```
Ans = 0;
for(int i=1; i<n; i++){
        for(int j=i+1; j<=n; j++){
                Ans = max(max, p[i] + s[j]);
        }
}
```

T.C = O(N*N), S.C = O(N+N) = O(N)

Optimized :

```
n
a[n+1]
pre[n+1];
for(int i=1; i<=n; i++){
        pre[i] = max(a[i], a[i] + pre[i-1]);
}
```

Now let's calculate the suffix array

```
s[n+1]
```

max_s[n+1] ⟹ It will tell the max sum of a subarray among n-1 to i

s[j] ⟹ it will tell us that max possible subarray sum starting at index j

```
 for(int i=n-1; i>=2; i++){
```

```
        s[i] = max(a[i], a[i] + s[i+1]);
        max_s[i] = max(s[i], max_s[i+1])
}

Ans = 0;
for(int i=1; i<n; i++){
        int sum = p[i] + max_s[i+1];
        Ans = max(ans, sum);
}
```

Real Question :

The second subarray should be increasing

Ex : [5 4 3 1 8 10 1 2]

Length of Increasing subarray starting at index " i " →[1 1 1 3 2 1 2 1]

s[i] → It will tell us the longest increasing subarray sum at index i

s[] = [5 4 3 19 18 10 3 2]

u[] = [19 19 19 19 18 10 3 2]

max[] = [19 19 19 19 18 10 3 2]

Ex : [5, 4, 3, -1, -8, -10, 1, 2]

Length of Increasing subarray starting at index " i " →[1 1 1 5 4 3 2 1]

s[] = [5, 4, 3, -16, -15, -7, 3, 2]

u[] =[5, 4, 3 , -35, -19, -4, 3, 2]

n

a[n+1]

pre[n+1];

```
s[n+1]
max_s[n+1]
for(int i=1; i<=n; i++){
        pre[i] = max(a[i], a[i] + pre[i-1]);
}
```

Now let's calculate the suffix array

max_s[n+1] ⟹ It will tell the max sum of a subarray among n-1 to i

s[j] ⟹ it will tell us that max possible subarray sum starting at index j

```
 for(int i=n-1; i>=2; i++){
        s[i] = max(a[i], a[i] + s[i+1]);
        max_s[i] = max(s[i], max_s[i+1])
}
```

" राम - राम "

Link :

https://leetcode.com/problems/stable-subarrays-with-equal-boundary-and-interior-sum

Ex : [9 9 0 -9 9 3 3 3 9]

Valid : [9 3 3 3 9], [9 0 -9 9 3 3 3 9], [3 3 3]

Understanding : - We are given an array of size n and subarray [i ……..j] is stable when a[i]==a[j] and sum[i …….. j]== a[i]

Brute Force :

Analysis : Iterate through all possible subarrays and maintain a running sum and check if a[i] == a[j] and sum[i …….. j]== a[i]

T.C = $O(N^2)$

Code :

Optimization :

A subarray is valid only if sum[i ……. j] = 3*a[i]

We want to check the sum so we will calculate the prefix sum of the array so that whenever we are at index j then and want to check this subarray [i …… j] we can check if the

(pre[j] - 2*a[j]) exists in prefix array then we can say that it is possible that this subarray is valid but we also need to take care of val in arr at which prefix sum is present to check if a[i] = a[j] or not

If a[i] ≠ a[j] then subarray is not valid

“ राम - राम ”

Link-> [Atlassian OA | SDE 1 | Freshers Hiring | CTC 75 Lac - Desi QnA]

Understanding : We are given an array stock price[] of size n

We have to return the max sum of a subset such that stockprice[chosenday[i]] - stockprice[chosenday[i]] = chosenday[i] - chosenday[j]

Let’s say subset ⇒ (s[1], s[2], s[3], ……., s[t])

I.e we have a subset of indices and we have to check if

⇒ stockprice[i] - stokprice[j] = i - j;

⇒ ⇒ stockprice[i] - i = stokprice[j] - j;

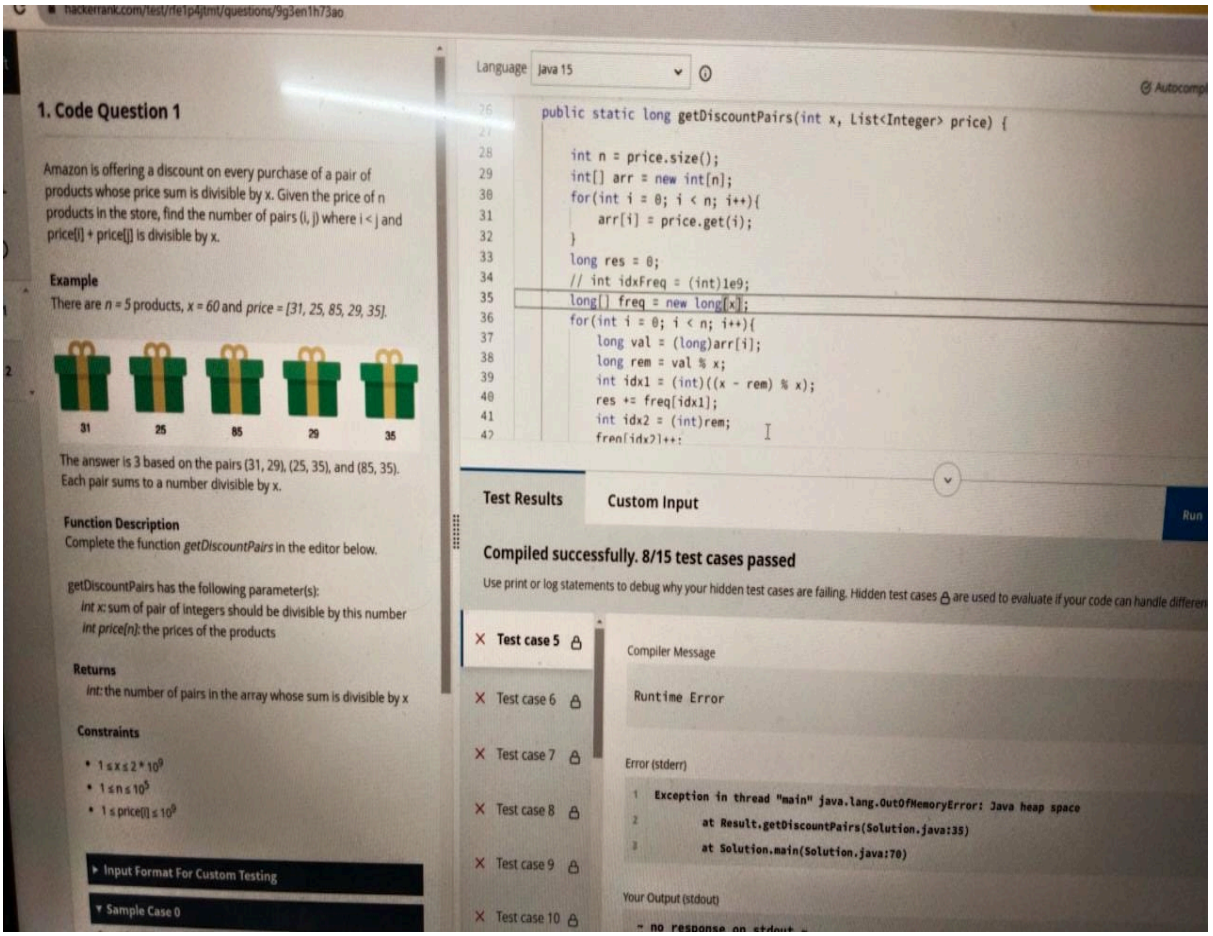
We will simply create a new array in which we will store the stockprice[i] - i at each index

Then simply use a hashmap to store stockprice[i] - i as a key and sum of all values who is giving stockprice[i] - i

" राम - राम "

Link :

https://www.designa.in/15334/amazon-sde-intern-ireland-australia-india-august-2023-coding



“ राम - राम ”

-> Given an array of size “N” ; find the maximum sum subarray which has the same elements in the start and end.

Example :- [1 8 10 8 -5 8]

O/p : [8 10 8 -5 8] = sum = 29

Brute Force :

```
int[] a = new int[n];
int max = 0;
for(int i=0; i<n; i++){
        int sum = 0;
        for(int j=i; j<n; j++){
                sum += a[i];
                if(a[i] == a[j]){
                        max = Math.max(max, sum);
                }
        }
}
print(max);
```

Optimized : Whenever we want to find the subarray[i ..... j] sum then we take that subarray and try to analyze it

In this we want to find the max subarray sum such that our start and end of that subarray is equal

So to fund subarray sum we can make of prefix[] array in which pre[i] = sum from 0 i

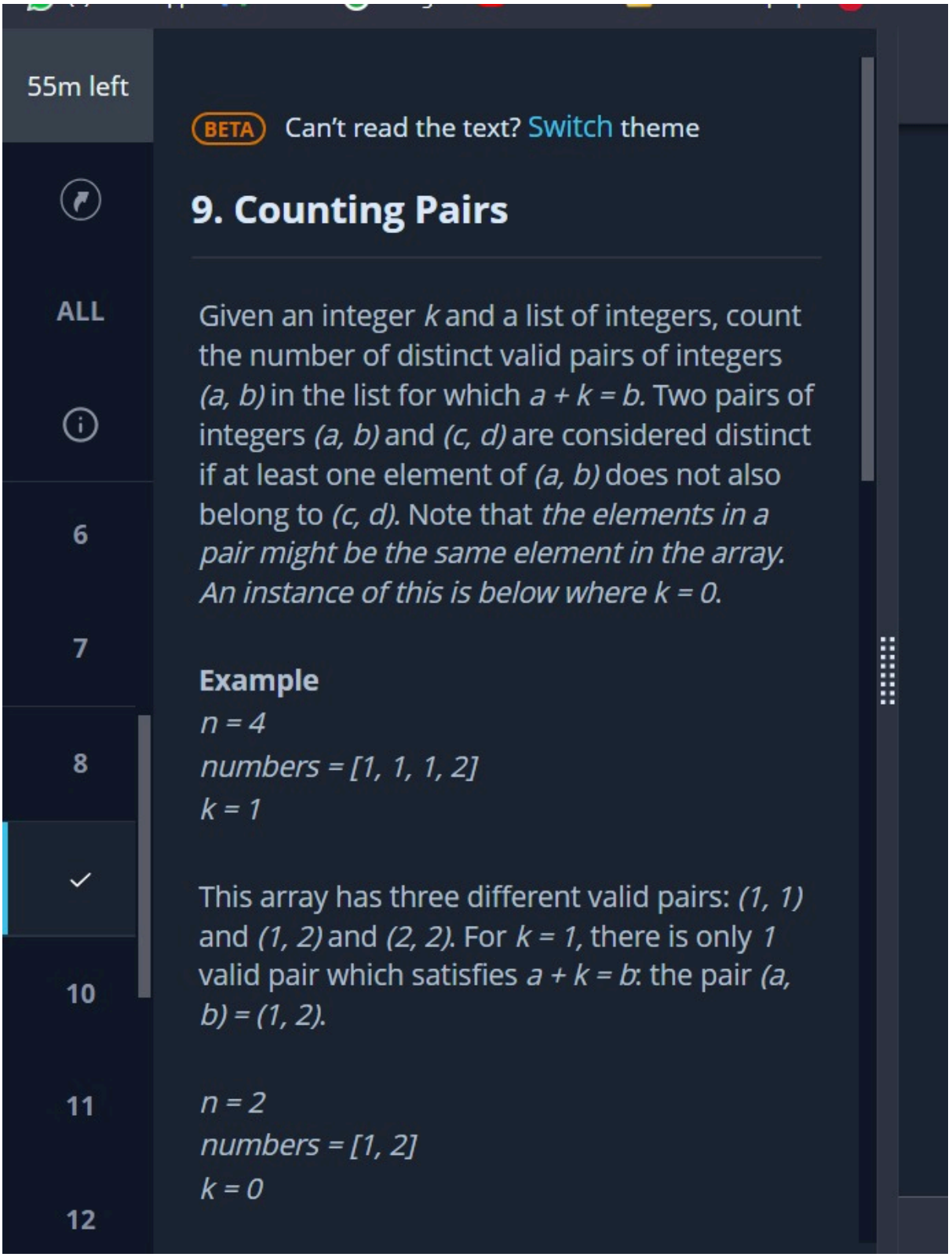
To track the element at index i we will make use of map

Map<a[i], pre[i]> map;

This pre[i] should be minimum for that a[i] so that we can get the max sum

Code : https://onecompiler.com/java/45697v9jg

" राम - राम "



Understanding :

We are given a list of size n and integer k , we have to count the number of distinct valid pairs (a,b) such that a+k = b

Code : https://onecompiler.com/java/456e3zua9

" राम - राम "

Main Question :

Given an array of size "N"; find the maximum value of triplet (i,j,k) such that |a[i] - a[j] - a[k]| + |i-j-k| is maximum.

Easier Version :

Find max value of pair (i, j) such that |a[i] - a[j]| + |i-j| is max

Analysis : when we open absolute we get
If (x > y) |x – y| = x – y
(y > x) = –x + y = y – x

So for this equation : |a[i] – a[j]| + |i – j|
Four possibilities : (+, +), (–, +), (+, –), (–, –)
1 ⇒ a[i] – a[j] + i – j = (a[i] + i) – (a[j] + j)

2 ⇒ -a[i] + a[j] + i – j = (i – a[i]) + (a[j] – j)

3 ⇒ a[i] – a[j] –(i – j) = (a[i] – i) + (j – a[j])

4 ⇒ -a[i] + a[j] – i + j = (a[j] + j) – (a[i] + i)

If we see carefully 1 & 4 are same and 2 & 3 are same
So there are only two options remaining
1 and 4
ans1 = (Max)(a[i] + i) – (Min)(a[j] + j)
ans2 = Max(i – a[i]) + (Max)(a[j] – j)
finalAns = Max(ans1, ans2)

Real Question : |a[i] - a[j] - a[k]| + |i-j-k| is maximum.

Here i == j == k is also possible

Again 4 possibilities = (+, +), (–, +), (+, –), (–, –)

ans1 ⇒ (Max)(a[i] + i) - (Min)(a[j] + j + a[k] + k)

ans2 ⇒ (Max)(a[i] - i) - (Min)(a[j] - j + a[k] - k)

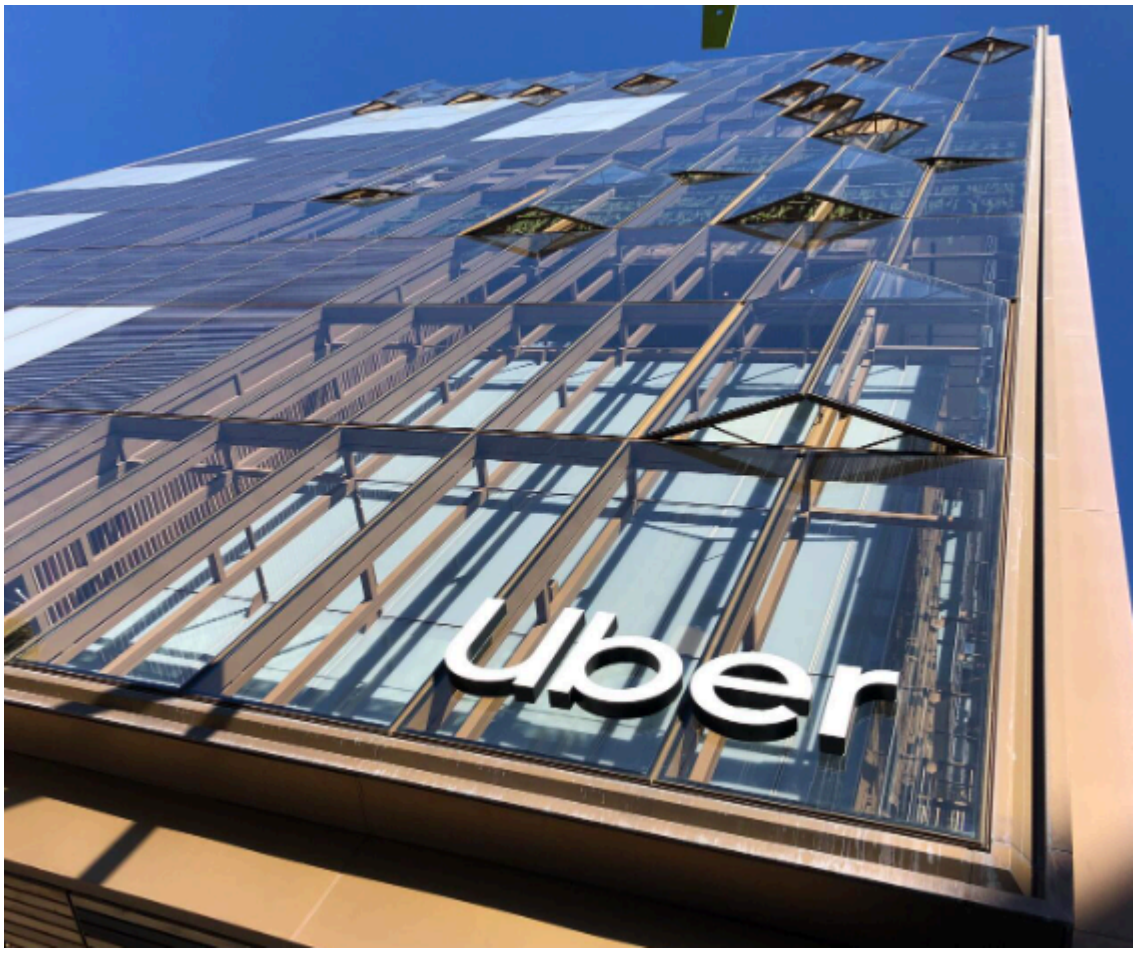
ans3 ⇒ (Max)(i - a[i]) - (Min)(a[j] - j + a[k] - k)

ans4 ⇒ (Max)(a[j] + j + a[k] + k) - (Min)(a[i] + i)

Ans = Max(ans1, ans2, ans3, ans4)

“ राम - राम ”

“ राम - राम ”



Find the number of subarrays such that their maximum is K -

Ex : [8 2 1 3 4 5 1 10] , K = 3

Output = 3. [3],[1 3],[2 1 3]

```
Brute Force :
Generate all possible subarrays and check if its max
is k then increment the count
T.C = O(N^2)
Int count = 0;
for(int i=0; i<n; i++){
     max = a[i];
     for(int j=i; j<n; j++){
          max = Math.max(max, a[j]);
          if(max == k)count++;
     }
};
```

Optimized :
If my current number is k then number of subarrays containing k is x*y + (x+y) + 1
where x is count of numbers before k and y is count of numbers after k
[8 2 1 3 0 1 2 4 5 1 10]
In this example here count of numbers before k is 2 and after k is 3 which are making a subarray with k

So total subarrays for x = 2 and y = 3 containing k => 2*3 + (2+3) + 1 = 12
(2,1,3),(2,1,3,1),(2,1,3,0,1),(2,1,3,0,1,2),(1,3),
(1,3,0),(1,3,0,1),(1,3,0,1,2),(3,0),(3,0,1),
(3,0,1,2),(3)

When we have an array like this [8 2 1 3 0 1 10 3 1 2]

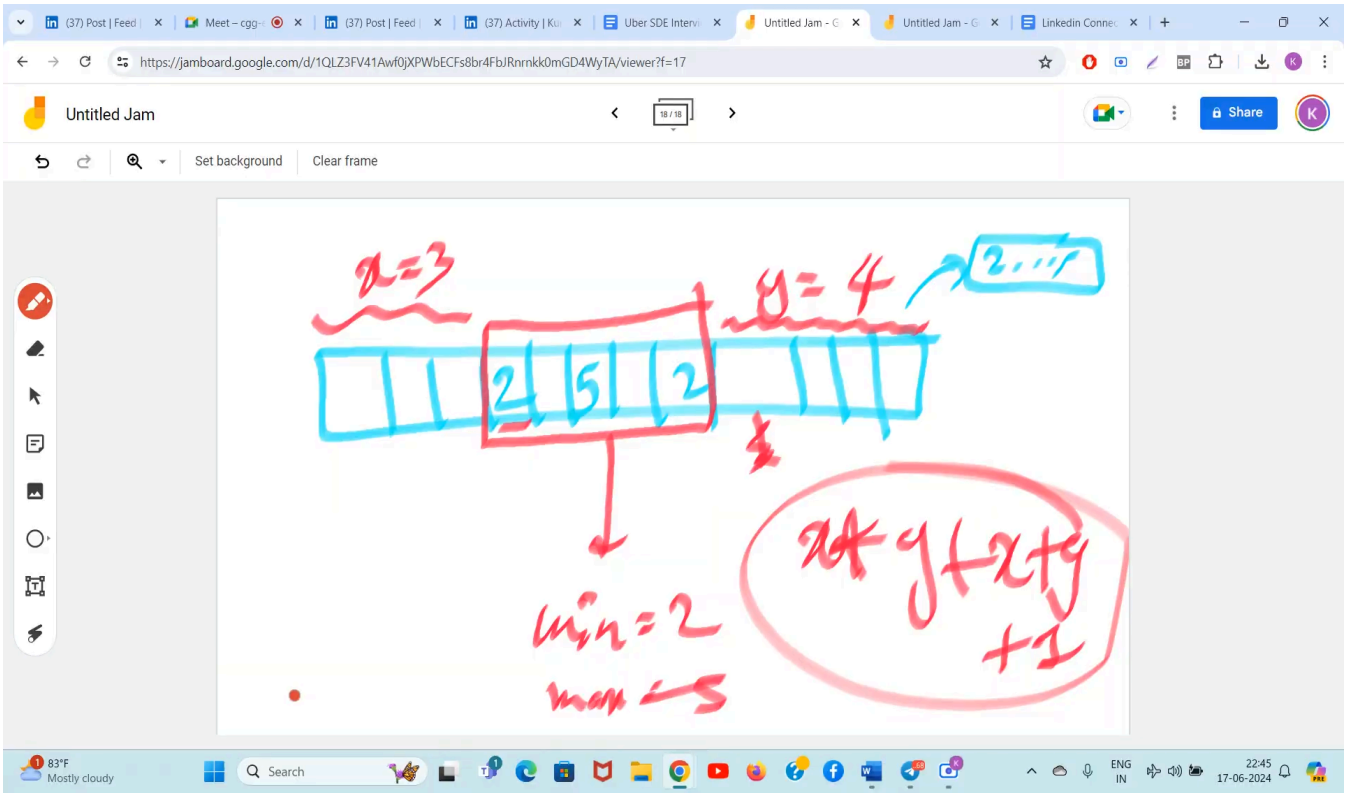
Then also our formula will work b'coz count of x and y on left and right resp. of k does not include k so here there will be no repetition of count
So k at index 4 has contribution of 2*2 + 2 + 2 + 1 to total count And now contribution of k at index 8 has contribution of 0*2 + 0 + 2 + 1
So total number of subarrays having k as their max = 9+3 = 12

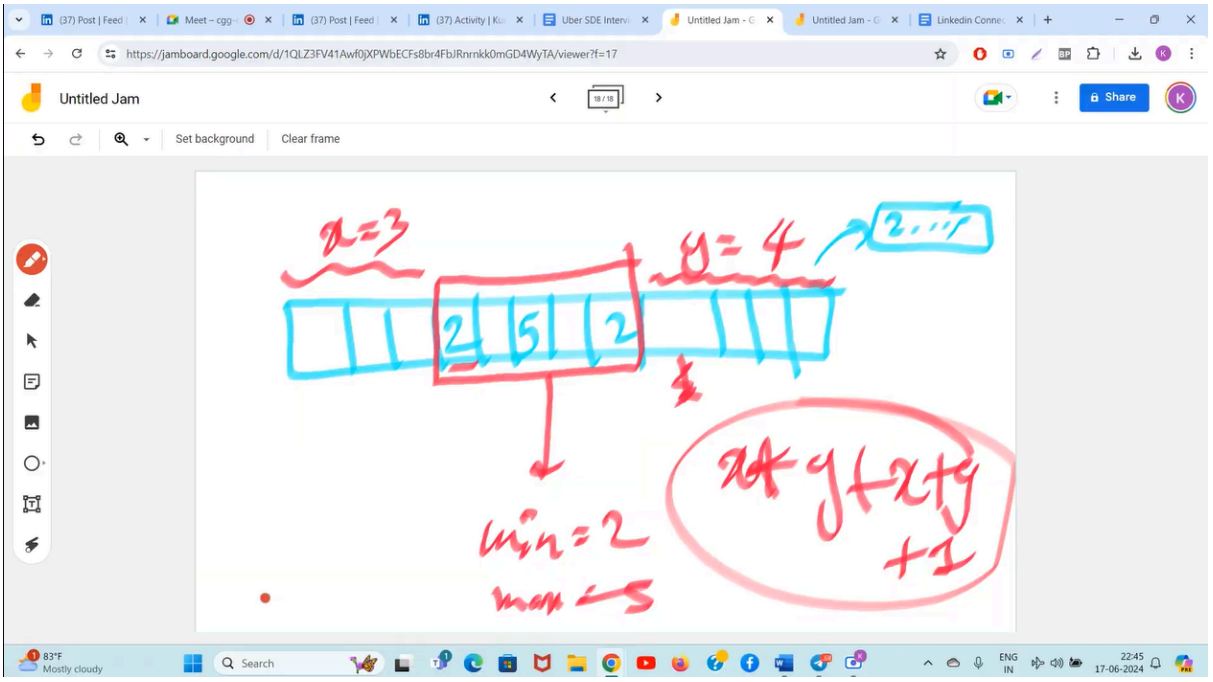
When we have an array like this [8 2 1 3 0 1 3 1]
Now if we apply that formula here then the count of subarrays will be
For k at
⇒ idx 4 = 2*4 + 2 + 4 + 1 = 15
⇒ idx 7 = 5*1 + 5 + 1 + 1 = 12
Total = 27
So to fix this what we can do is we will count all the numbers <= k from left and from right we will

count only those numbers < k that can make subarray with k
Hence it will fix the problem of repeated count
So corrected ans
⇒ at idx 4 = 2*2 + 2 + 2 + 1 = 9
⇒ at idx 7 = 5*1 + 5 + 1 + 1 = 12
Total = 21

Find the number of subarrays having min as k1 and max as k2

First try to find the good subarrays
A subarray is good when we can say that this subarray has min as k1 and max as k2
Ex : [] , k1 = 2, k2 = 5




Untitled Jam
https://jamboard.google.com/d/1QLZ3FV41Awf0jXPWbECFs8br4FbJRnrnkk0mGD4WyTA/viewer?f=17
Share
Set background
Clear frame
x=3
y=4
2 5 2
min=2
max=5
x+y+x+y
+1